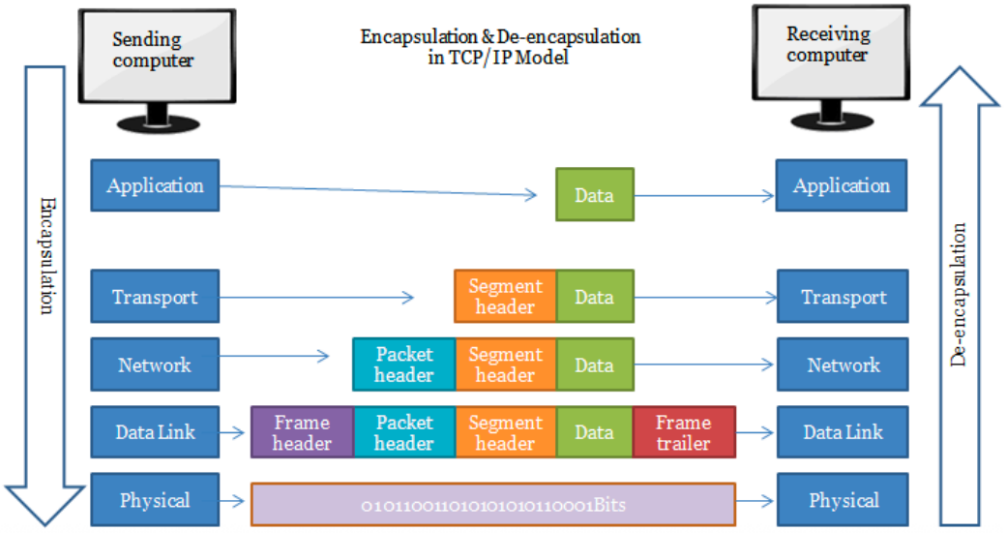
OSI Layers



1. Physical (Binary, Physical Signals)

- Wi-Fi, Bluetooth, Ethernet
- lshw -class network (**l**ist **h**ard**w**are)
  - Shows networking hardware
  - Shows Controllers and Interfaces
    - Interfaces are children of controllers
    - Interfaces provide logical name of the interface
      - **eth** = ethernet
      - **wl/wlan** = WiFi
      - **vi/vir** = virtualized network devices
- **ifconfig** shows the status of network interfaces
  - **MTU** (Maximum Transmission Unit) = max packet size
  - **RX** (Received packets)
  - **TX** (Transmitted Packets)
  - Shows how many packets were dropped or produced errors
- **Debugging** with **lshw** and **ifconfig**
  - Identify controller **bus type** – **PCI** or **USB**
  - Identify interface capabilities – **ethernet** or **wireless**
  - Identify **manufacturer** and **product name** of controllers
  - Finding if a network adapter has been properly detected by Linux (if not, you may need to install drivers)

- Checking for idle or unexpected network interfaces
- Finding:
    - Dropped packets (**dropped**)
    - Misaligned frames (**frame**)
        - Bits at the start of a frame are aligned so they can be properly read
    - Overrun buffers **(overruns)**
    - Failed checksums/corrupted (**errors**),
    - Wireless errors like **carrier** (modulation failure) and **collisions** (interfaces transmitting at the same time)
        - Carrier means data cannot be converted to a radio wave properly
            - Usually means bad WiFi

2. Data Link (Frames, MAC addresses, Local Network Communication)

- Responsible for packaging **packets** from layer 3 into **frames**
- **Packets = Layer 3, Frames = Layer 2**
- Responsible for **physical addressing**
- Media Access Control (**MAC**) **address**
  - All internal (non-loopback) interfaces have a **48-bit** MAC address
  - Frames are addressed to specific MAC addresses
- Address Resolution Protocol (**ARP**)
  - Translates MAC addresses into IP addresses
    - Use MAC's to identify devices as they are static unlike IP's
  - **arp -n** will output the current arp table
    - Broadcast Request (Who has this MAC?)
    - Unicast Response (Here's my IP)
-

Universal/local and individual/group bits in MAC addresses

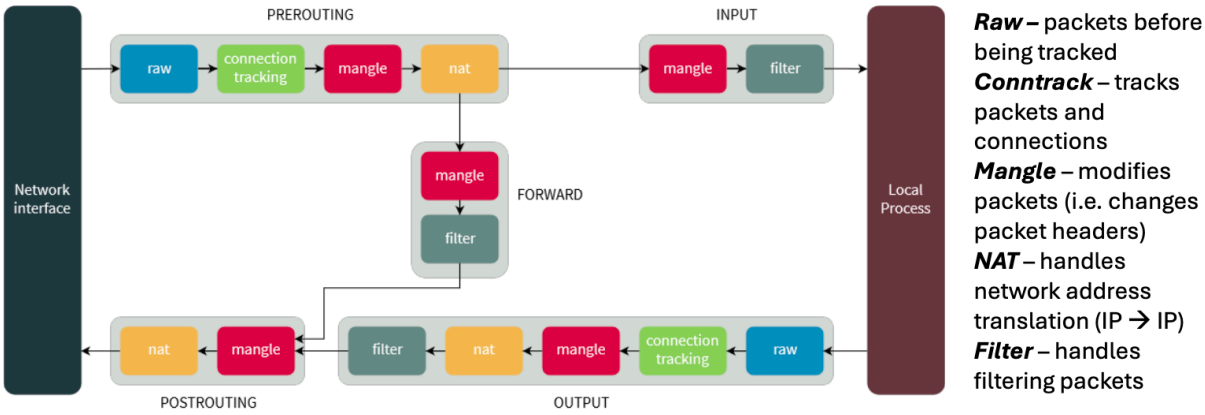
I/G \ U/L	Universally administered	Locally administered
Unicast (individual)	X0-XX-XX-XX-XX-XX X4-XX-XX-XX-XX-XX X8-XX-XX-XX-XX-XX XC-XX-XX-XX-XX-XX	X2-XX-XX-XX-XX-XX X6-XX-XX-XX-XX-XX XA-XX-XX-XX-XX-XX XE-XX-XX-XX-XX-XX
Multicast (group)	X1-XX-XX-XX-XX-XX X5-XX-XX-XX-XX-XX X9-XX-XX-XX-XX-XX XD-XX-XX-XX-XX-XX	X3-XX-XX-XX-XX-XX X7-XX-XX-XX-XX-XX XB-XX-XX-XX-XX-XX XF-XX-XX-XX-XX-XX

- First 3 octets of a MAC address identify the manufacturer
- You can use **ifconfig** to change the MAC of a network interface
  - **ifconfig <interface name (eth1)> down**
    - Shuts down the specified interface
  - **ifconfig eth1 hw ether 02:03:DE:AD:BE:EF**
    - Sets a new MAC on the interface
    - You can't set the first octet and a multicast address
  - **ifconfig eth1 up**
    - Reenables the interface
  - Not persistent across reboots
  - To permanently change it, you will need to edit the **/etc/networks** file
- **ebtables**

    - Provides a form of frame filtering based off physical addresses. These rules are only applied to a Bridge Interface = Mapping an inbound network interface to an outbound one (Using a raspberry pi to strip ad frames before forwarding to your TV)
    - **ebtables -L**
        - Gives you the rules for the current ebtable
- **Netfilter**
    - Kernel framework that handles networking in Linux
- **Arptables**
    - Allows filtering of ARP traffic. Good for preventing malicious ARP responses
    - **arptables -L**
        - Gives you the rules for the current arptable
        - Allows you to filter Layer 2 (ARP) and 3 (IP) traffic

3. Network (Packets, IP addresses)

- Deals with **packets**
  - Addressed using IP addresses
  - Network Address Translation (**NAT**)
  - Packaging data into **packets** using **IPv4** or **IPv6**
  - Making sure packets arrive intact and at the correct IP
- **ifconfig**
  - **inet (IPv4)** and **inet6 (IPv6)** address of each interface are provided
  - Includes **netmask** (subnet size)
  - **Broadcast IP address** as well sometimes
- **netstat**
  - Shows current active **Layer 3** connections
  - --inet flag limits to **IPv4** (use --inet6 for **IPv6**)
  - -n flag forces it to provide IP addresses
- **iptables/ip6tables**
  - Acts as the primary firewall in Linux
  - Determines which **ports, Ip's, protocols,** and **MAC's** are allowed to connect to the server



  - iptables –table <tablename> --list
    - Shows rules for a specific table
- Tables in IP Tables
  - **RAW**
  - **NAT**
  - **FILTER**
  - **Mangle**
  - **Security**

- Blocking an IP
  - **iptables -A INPUT -s 209.53.30.88 -j DROP**
    - Applies the rule to the INPUT chain (-A INPUT)
    - Impacts the IP Address (-s 209.53.30.88)
    - Set the target to DROP (-j DROP)
- Blocking a range of IP's

iptables -A INPUT -m iprange --src-range 209.53.30.70-209.53.30.90 -j DROP

- **iptables -L -v**
  - Gives you info about what IP's are banned on each chain, and the # of packets and bytes are being blocked
- Rules added with iptables commands are **TEMPORARY**
- **iptables -F (Flush)** resets all tables back to empty

iptables -A INPUT -m iprange --src-range 209.53.30.70-209.53.30.90 -j DROP

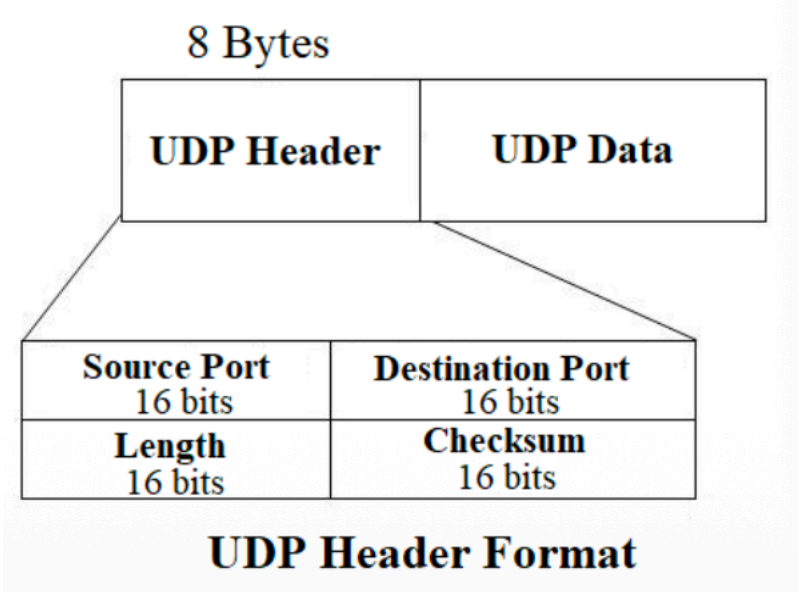
- **Packet Filtering**
  - Packets can be selectively **dropped, accepted, rerouted, or modified** based on predefined rules
    - Accepted packets are passed up to the next layer
    - Dropped packets are ignored
- **Uncomplicated Firewall (UFW)**
  - Relies on iptables
  - A command line utility to open and close specific **ports, protocols**, or **applications** for traffic
  - ufw allow 'NGINX Full' (Port 80, 443)
  - ufw allow 'NGINX HTTP' (Port 80)
  - ufw allow 'NGINX HTTPS' (Port 443)
  - Uses **Stateful Filtering**: Outbound connections initiated by the server are automatically allowed
  - Can be set for UDP or TCP
    - **sudo ufw allow 53/udp**
    - **sudo ufw allow 53/tcp**
- **Fail2Ban**
  - Automatically reads through the log files of softwares like OpenSSH or NGINX, and looks for evidence of bad behaviour
  - If a bad actor is spotted, Fail2Ban will add rules to the iptables to ban that specific IP for some set amount of time
- **ICMP (Internet Control Message Protocol)**

- Used for debugging and diagnostic purposes
- Share the same underlying packet format as IP, but are handled and processed separately
- **Ping <ip address / domain name> -c count**
    - Pings a given IP address, and sends **count** number of pings
- **Traceroute**
    - Tells you what servers/routers exist between us and some destination address (**route**)
    - Uses UDP by default
    - Parallels **ARP**, provides information on neighbours for its respective layer
    - **sudo apt install traceroute**
        - Installs traceroute
    - **traceroute -I -m 25 <ip / domain name> >> myfile.txt**
        - **-I** forces an **ICMP** message instead of **UDP**
        - **-m 25** Sets maximum number of hops to 25
        - **>> myfile.txt** pipes output to a file (Not required)

4. Transport (Ports)

- Responsible for transmission protocols
- **Transmission Control Protocol TCP**
    - Connection Oriented
    - Packet Ordering
    - Resending Missing Data
    - Handshakes
    - Maintaining Connections
    - **SSH**
- **User Datagram Protocol (UDP)**
    - Connectionless
    - No Handshake behaviour
    - **Security Concern:** No method to determine if a packet came from its source address
    - **NTP (Network Time Protocol)**
        - Uses UDP messages sent from your machine and time servers to receive accurate clock time from an primary source
        - **timedatectl**
            - Displays current system clock information

- **ss – tcp – all**
    - Displays information about current TCP connections
- **ss -tulpn**
    - **t**cp
    - **u**dp
    - **l**istening sockets
    - **p**rogram PID that owns the port
    - **n**umeric IP addresses
    - Prints open TCP and UDP, and local address as well as the port that is being used
    - **UCONN = UDP**
    - **LISTEN = TCP**
- **dig <domain>@<ip>**
    - sends DNS requests to the specified DNS resolver
- **tcpdump**
    - Captures incoming or outgoing UDP or TCP traffic on any interface or port
    - Includes source address, destination address, DNS packet ID, and data about the packet
- **sudo timeout 30 tcpdump udp port 53 --print > dns_dump.txt &**
    - Captures 30 seconds (**timeout 30**) of UDP traffic on port 53, parse and dump it in into **dns_dump.txt** in the background (**&**)
- **Ports**
    - Provide us a way of organizing traffic and managing connections
    - A network connection can be defined as:
        - IP address (Layer 3) + Transport Protocol (Layer 4) + Port Number (Layer 4)

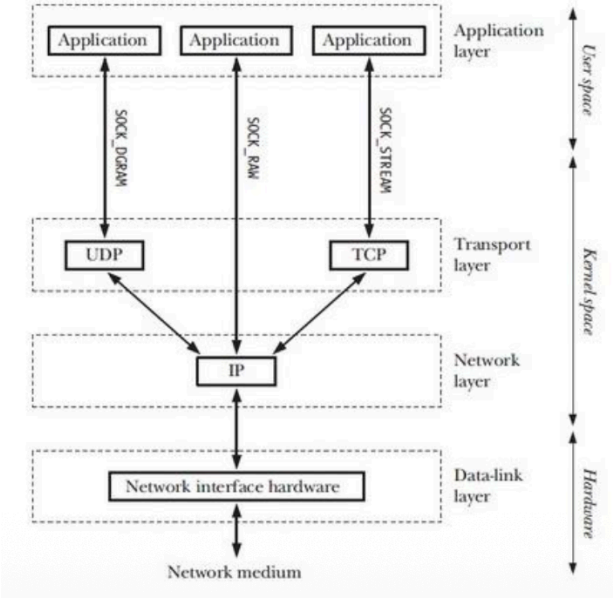


- **16 bit Number (0 - 65535)**
  - **0 – 1024** = well known ports (Associated with specific services)
    - **SSH** = 22
    - **DNS** = 53
    - Client is usually connected with a **randomly assigned ephemeral port**
  - **1025 – 65535** = ephemeral ports (disposable, temporary)

5. Session (Sockets)

- **Sockets**
  - The entity that manages a port and handles the receiving and sending of data on that port
  - Bridge the **kernel** and **user** space
  - **Not Responsible** for any **security** or **data integrity**
  - Only have the information presented by the transport/IP layer for client addressing
    - For servers that have proxy servers between them and the client, the socket will report the proxy as the client address
  - Responsible for maintaining and managing persistent connections
  - A connection to a network interface, bound by an **address** and a **port**, which returns a **file descriptor**
    - The file descriptor allows the socket to be handled like a file
  - Sockets can appear in two different forms:
    - **Network Sockets:** Usually used for IPv4 and IPv6 but not always

- Three types of Network Socket:
  - **Streaming:** TCP. Persistent connection
  - **Datagram:** UDP. Non-persistent
  - **Raw:** Skip most OSI layer processing, allowing direct access to raw layer 3 protocol
- **Unix Domain Sockets:** Used only internally, to allow transfer of data between two local processes (local socket)



- **Life Cycle of a Streaming Socket**
  - Starts with a bind, attaches the socket to a network address and port

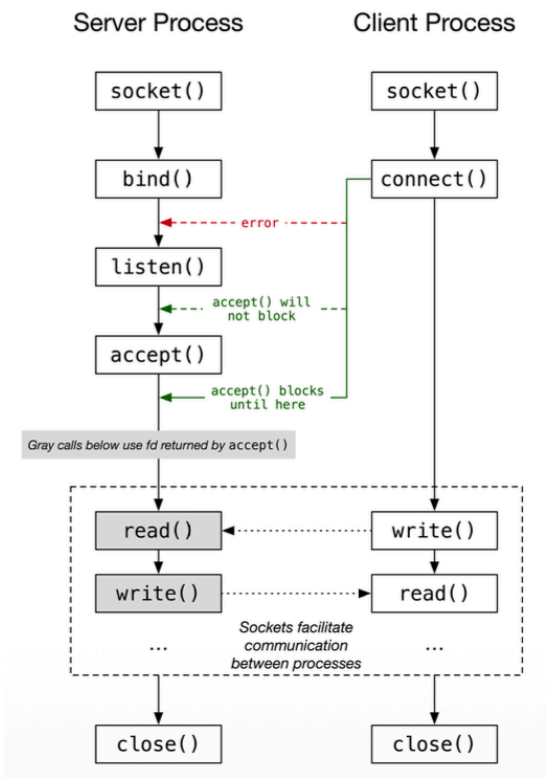
```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(('0.0.0.0', 25000)))
```

  - The socket will then listen for traffic on the port, and accept new connections from clients

sock.listen(5)

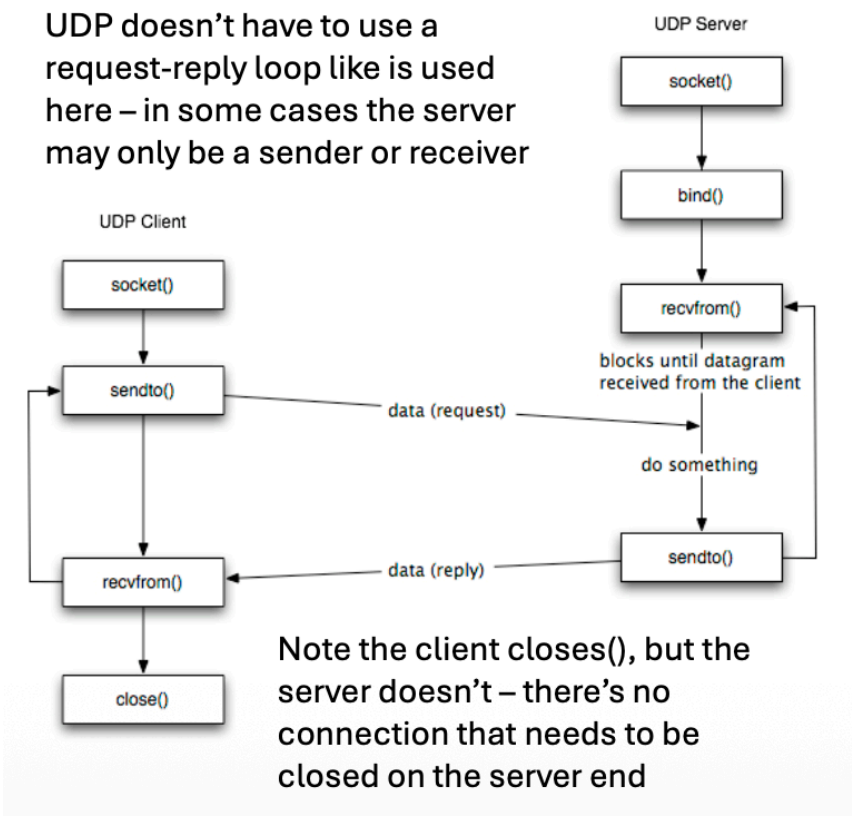
sudo ufw allow 25000/tcp (Opens port 25000 for TCP)

  - Once connection is established, the socket can read or write data to the clients socket

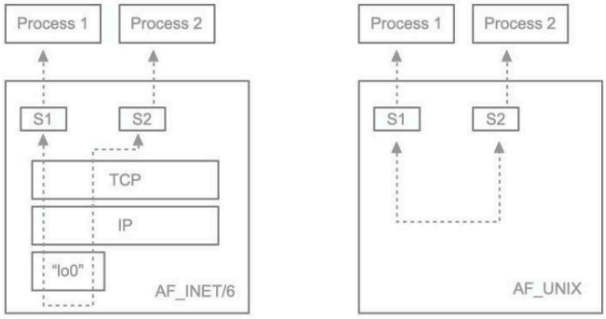


- When the **accept()** function is called, a **session** is started for the client, and the **socket** provides a connection objects that's used to communicate with the client
- **recv(maxbuffersize)**
  - Wait for up to maxbuffersize bytes to arrive from the client (for **raw** sockets, the input arrives one byte at a time)
- **send(buffer)**
  - Send bytes back to the client

- **Datagram Socket Lifecycle**
  - Simpler than a Stream socket
    - No accept to close on the server side (connectionless)
    - The server just binds the socket to an address and port, and receive any datagram that arrives on that port
    - It may also send UDP data back to the client



- **Unix Domain Sockets**
    - Not connected to the internet
    - Means of allowing two local applications to talk to each other
    - More efficient than a network socket



6. Presentation (Encryption, MIME)

- Encoding and data parsing
- The layer that handles encryption
- **TLS**
    - Provides end-to-end encryption and packet signing
    - Important for securing other protocols like HTTPS, SMTP and NTS (Secure NTP)
    - **openssl version**
        - Displays the current ssl version
    - **openssl s_client – connect <domain, ip> -tls1_3**
        - Connects to a server with TLS encryption
    - In Reality Layer 6 and 7 can sort of blend together
    - Layer 6 elements of HTTP (Layer 7)
        - **Content-Encoding** is **compression** (uncompressed or compressed with a variety of schemas)
        - **Content-Type** tells the client how to parse the content
            - **text/html**
                - Optional **charset**
            - **image/gif**
            - **video/mp4**
        - **Transfer-Encoding**: how data is passed between two nodes – it is almost always **chunked**
            - Data is transmitted in chunks with length bytes prefixed on and then reassembled

7. Application (SSH, HTTP)

- **SSH**
- **DNS**
- **NTP**
    - **timedatectl timesync-status**
        - Provides the IP of the time server were connected to
        - Timekeeping information (delay, jitter, precision etc.)
    - To change time servers:
        - **sudo nano /etc/systemd/timesyncd.conf**
        - **NTP=time.nrc.ca**
    - NTP Refresh
        - Disable NTP

            - **sudo timedatectl set-ntp false**
        - Change the time
            - **sudo timedatectl set-time "2026-01-01 00:00:00“**
        - Re-enable NTP
            - **sudo timedatectl set-ntp true**
    - Format
        - The first **byte** of the message sets the **LI** (Leap Year), **VN** (Version), and **Mode** (Client or Server) bits
            - **LI (2 bits)**
                - 0 = no warning
                - 1 = last minute has 61 seconds
                - 2 last minute has 59 seconds
                - 3 = unknown (used for leap year correction)
            - **VN (3 bits)**
                - Represent the version of NTP being used (usually **version 4**)
            - **Mode (3 bits)**
                - 3 = client
                - 4 = server
            - The rest of the message (47 bytes) is all zeroes
        - **Unix Timestamp uses Jan 1, 1970, While NTP uses Jan 1, 1900**
        - The actual timestamp value we want (the transmit timestamp) is packed in with a bunch of other timestamps so we need to unpack it properly
        - Timestamp fields are 64 bit and split into 2 32-bit halves
            - First half is integer seconds
            - Second Hlaf is fractions of a second
- **HTTP(S)**

Server Setup

- **apt update && apt upgrade -y**
    - Updates and install all packages
- **dpkg-reconfigure -plow unattended-upgrades**
    - Lets you configure unattended upgrades

Systemd Timers and Services

- Check timers with: **systemctl list-timers**
- apt-daily (update) and apt-daily upgrade (upgrade)
    - Update runs first by default
- Use **chron** for scheduling tasks
- **systemctl edit apt-daily.timer**
    - opens an overriding configuration file
    - **OnCalendar**
        - Has the format Has the format Y-M-D H:M:S
        - **OnCalendar=*-*-* 22:00:00** will set it to update everyday at 10pm
    - **RandomizedDelaySec=1800**
        - Adds a randomized delay between 0 and 1800 seconds to the scheduled time
    - **After=apt-daily.timer**
        - Makes sure a timer follows another timer
- **systemctl edit --force --full reboot.service**
    - Creates a new service
- **systemctl enable –now <name>.timer**
    - Enables a timer
- **systemctl list-timers**
    - Lists all current timers on the system
- **systemctl daemon-reload**
    - Refreshes timers if any changes have been made to them
- **timedatectl set-timezone America/Vancouver**

Swap Space

- Acts as a slower RAM expansion
- Uses allocated disk space to function as extra ram
- **fallocate -l 1G /swap**
    - Creates 1GB file on the root partition called swap
- **chmod 600 /swap**
    - Allows only **root access** to the swap file
- **mkswap /swap**
    - Specifies the file to be used for swapping (swap)
- **swapon /swap**
    - Enables the swap
- **free -h**
    - Displays the current RAM and Swap space
- **echo '/swap none swap sw 0 0' | sudo tee -a /etc/fstab**

    - Makes the swap permanent

Server Hardening

Create New User

- **useradd -m -g sudo <username>**
    - Adds a new user
- **passwd <username>**
    - Sets password for a user
- **su <username>**
    - Switches to a different user

Setting up an SSH Key

- Generate Key with PuTTY or SSHKeygen
- Create the .ssh directory with **mkdir ~/.ssh**
- Create key file with **nano ~/.ssh/authorized_keys** and paste the key into that file

Lock the Root Account

- **sudo passwd -l root**
- To recover the account, you can set a new password with **sudo passwd root**

Changing SSH Port and Enabling Key Login Only

- **sudo nano /etc/ssh/sshd_config**
    - Change the Port 22 line to a different port of your choosing
    - Change the **PermitRootLogin** from **yes** to **no**
    - Uncomment the **PubkeyAuthentication yes**
    - Change the **PasswordAuthentication yes** line to **no**
        - Disables Users logging in with password
    - **sudo systemctl restart ssh**
        - Restarts ssh
    - You also need to change a server generated overriding file:
        - **/etc/ssh/sshd_config.d/50-cloud-init.conf**
- **sudo ufw status**
    - Displays the current firewall status
- **sudo ufw default deny incoming**
    - Denies all incoming traffic

- **sudo ufw allow 1337/tcp**
    - Allows traffic on port 1337
- **sudo ufw show added**
    - Shows added firewall rules
- **sudo ufw enable**
    - Enables the firewall

SSH Audit

- **sudo apt install ssh-audit**
    - Installs SSH audit
- **ssh-audit –p 1337 127.0.0.1**
    - Runs an audit on the opened port on localhost for weaknesses
- Change key algorithms in **sshd_config**

Fail2Ban

- **sudo apt install fail2ban**
    - Installs Fail2Ban
- **fail2ban-client –version**
    - Verifies installation and version

Creating a jail

- **sudo nano /etc/fail2ban/jail.local**

```
[ssh]
enabled = true
port = 1337
filter = sshd[mode=aggressive]
logpath = /var/log/auth.log
maxretry = 3
findtime = 5m
bantime = 5m
```

- **Attributes**
    - Name = [ssh]
    - enabled = true (Jail is enabled)
    - port = 1337 (Port it listens on)
    - filter = sshd[aggressive] (Uses sshd filter rules, and aggressively flags violations)
    - logpath = /var/log/auth.log (Location of the log file Fail2Ban is checking)

    - maxretry = 3 (How many violations within findtime before banning)
    - findtime = 5m (Length of sliding window for maxretry)
    - bantime = 5m (Length of ban time)
- **sudo systemctl enable fail2ban**
    - Automatically starts Fail2Ban when server boots reboots
- **sudo fail2ban-client status <jailname>**
    - Gives you info on # of banned IP's, total bans, list of IP's
- **sudo service fail2ban restart**
    - restarts fail2ban
- **Fail2Ban** can be very useful for Apache and NGINX web servers
    - Has detection for SQL Injection, scrapers, and abuse of HTTP/S authentication

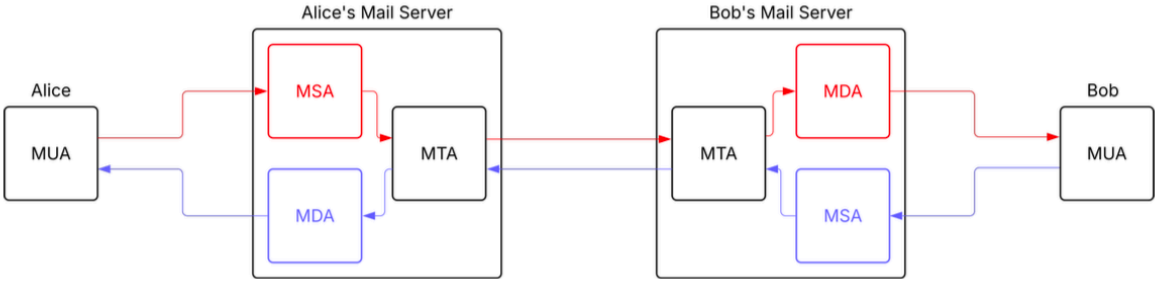
Port Scanning

- SSH servers send a **hello banner** to clients which contains **version information**
- Any **TCP** socket with a **public port** can be identified by port scanning
- Even though it's called port scanning, its more **socket scanning**
    - A port scan will tell you what sockets respond to what protocols (**UDP** or **TCP**) so were getting info from layer 4 and 5
- **Types of Port Scanning**
    - **Full-handshake**
        - Port scanner engages in a **full TCP handshake** with the server
        - Once the first data packets are received from the server it disconnects
        - **Banner Grabbing**
            - Grabs the first data, typically a banner that identifies the type of service
    - **SYN Scanning**
        - Only sends the initial TCP **SYN** packet, wait for **SYN/ACK** response from server.
        - If **SYN/ACK** is received or the response times out, the scanner resets the connection
        - Increased speed and lower network usage when compared to Full-handshake
    - **Port Scanning doesn't have to be malicious**
        - Can't open a connection to your local Linux server? Do a port scan to verify open ports
        - Use a port scan to check if you've left any ports open by mistake

- ## NMAP
    - Uses full-handshake scanning by default
    - **nmap <domain name, IP address>**
    - **-p** (What ports to scan)
        - **-p1000-2000** scans ports 1000 to 2000
        - **-p1000-** scans ports 1000 to 65536
        - **-p-** scans all ports
    - **-A** aggressive port scans
        - More noticeable, take longer
        - Further software identification
            - Software versions
            - SSH hostkeys
            - HTTP content
            - OpenSSH can potentially leak OS info as well
    - **-T** timing of the scans
        - **-T0** (slowest) to **-T5** (fastest)
    - **-sU** Scan for open UDP ports
        - Relies on the server sending an ICMP response that a port is closed when receiving a datagram
        - Firewalls and UDP servers can cause nmap to return false positives
    - Scanning localhost will include services only exposed on localhost as well as other normal services
        - It can miss services bound only to external interfaces
- ## Problem with Port Scanning
    - Anything we do to protect ourselves from port scanning is likely to also prevent us from connecting, or limiting who can access the server
    - If you want no port scanning, you'd have to enable ufw to only allow whitelisted addresses through, which prevents general users from accessing the server
- ## Logging
    - **ufw** keeps a log of all rejected connection attempts
        - **/var/log/ufw.log**
- ## Active Intrusion Detection
    - You can write a regex to detect failed connections and capture the source IP

```
[Definition]
failregex = .*\[UFW BLOCK\] IN=.* SRC=<HOST>
```

- **Linux Email**
  - An **asynchronous** protocol
    - Sender and receiver are not always online at the same time
  - **MUA** (Mail User Agent)
    - Take email traffic and render it in an **organized** and **intelligible** manner for the **client.**
    - Fetches mail from the **MDA**, typically using protocols like **SMTP** (Simple Mail Transfer Protocol) or **POP3** (Post Office Protocol)
    - Provide Authoring tools to create email messages
    - Provide the initial sending protocols to get messages off the client and into the network
    - Common **MUA's** include Outlook and Thunderbird
  - **MTA** (Mail Transfer Agent)
    - Accept emails from **MUA's** or middlemen called **Mail Submission Agents** (MSA) which pass mail from **MUA's** to **MTA's**
    - **MTA's** use a mail relay protocol such as SMTP to send mail to the next **MTA**.
      - Mail will travel between **MTA's** until it reaches the **MTA** with the recipients **MDA** attached to it
      - The mail will then stay in the **MDA** until the **MUA** downloads it
  - **MDA** (Mail Delivery Agent)
    - Servers as a storage space for mail
    - Acts as a centralized inbox for one or more users (typically for a domain)
    - Acts as a **buffer** for the mail system
    - **MUA's** will connect to the **MDA** and download available emails until they are **synchronized**



  - The **MTA** and **MDA** often live on the **same mail server**

- **Mailutils**
    - **sudo apt install mailutils**
    - **Postfix = MTA**
    - By Default, your mail name is the hostname of the server
    - **Postfix released in 1998**
- **Spools**
    - Where new local email messages are held until they are read
    - New emails come in in raw MIME email format, which can be difficult to parse
    - To check the spool use the **mail** command
- Writing a local email
    - **echo "Hello World!" | mail -s "My Cool Subject" frit**
    - **OR mail frit**
- **SMTP**
    - **TCP-based** used for:
        - Forwarding a message to be sent from the client **MUA** to the **MTA**
        - Forwarding a message from **MTA** to **MTA** or from **MTA** to **MDA**
- **IMAP** (Internet Message Access Protocol) and **POP3**
    - Used to manage **inboxes**
    - Responsible for download emails from the **MDA** to the client's **MUA**
    - Responsible for deleting emails from the **MDA** to free up space, and securing the connection
    - **TLS** encrypted versions of **IMAP** and **POP3** are common to prevent unauthorized mailbox access
- **Life Cycle of an Email**
    - The end goal of the email system is to ensure that an email authored by an **MUA** will be handed to their **MTA** .
    - Then its passed through various **MTA's** until it arrives at the recipients **MTA**
    - The **MTA** then passes it to the recipients **MDA**
    - It will then be stored until the recipients **MUA** downloads it
- **How does the mail system know which MDA server is the correct one?**
    - **DNS** can be used with email domain names (@okanagan.bc.ca)
    - The **DNS query** is the responsibility of the sender's **MTA** server who will look up the recipients **MX** records
- **MX** (Mail Exchange) **Records**
    - Contain the **hostname,** which points to the mail server that the domain uses
- This poses a problem for our servers as they don't have ay **DNS** records set up, or a proper domain name

    - Technically you can send an email directly to an IP address like: user@1.2.3.4 but almost no mail servers will allow addresses like this to transit their systems (Security, Delivery Problems)
    - However, this only applies to the **receiving** side of email, we don't need a **domain name** or **MX** record to **send** email to an address
    - Using an **MTA** like **mailutils** in **send-only** configuration is quite common on Linux Servers
        - We care about sending mail more than we do receiving it
- Most **MTA** servers will include some sort of **spam filter** to identify scam/ and or spam emails and reject them automatically
    - A large part of spam filtering is **IP** and **domain reputation**: Big databases keep track of how trustworthy a given **IP** or **domain name**
    - If the previous owners of our IP were not being good, it can cause problems for us
        - We can ask the webhost to give us a new IP and hope the one they give us has a better reputation
        - Implementing encryption can boost your trustworthiness
- **SMTP Relays**
    - You can send mail to an **MTA** that isn't responsible for the given domain, instead that **MTA** can forward the email to the correct **MTA** server (**relay it**)
    - **Open Relay:**
        - A mail server (**MTA**) that accepts emails for **any domain name**
            - Generally Bad Business
            - Rife for abuse, spam, scammers etc.
        - Most mail servers will automatically ignore or spam any email arriving from an **open relay MTA**

```
[ssh]

enabled = true

port = 1337

filter = sshd

logpath = /var/log/auth.log

maxretry = 5

findtime = 5m

bantime = 5m
```